Greedy

4 min read Updated 16 Jul 2026 Topic 64 of 77

Bookmark Mark done

23G. Greedy (10 problems)

Pattern recognition: "At each step, pick the locally optimal choice and it leads to the global optimum."

23.81 Activity Selection / Meeting Rooms

JAVASCRIPT Copy

```
function maxActivities(activities) {
  activities.sort((a, b) ⇒ a.end - b.end);
  let count = 1, lastEnd = activities[0].end;
  for (let i = 1; i < activities.length; i++) {
    if (activities[i].start ≥ lastEnd) { count++; lastEnd = activities[i]
  }
  return count;
}
```

23.82 Jump Game

JAVASCRIPT Copy

```
function canJump(nums) {
  let farthest = 0;
  for (let i = 0; i ≤ farthest && i < nums.length; i++) {
    farthest = Math.max(farthest, i + nums[i]);
```

23.83 Jump Game II

Approach: Greedily track the farthest reachable index within the current jump's range; when you reach the end of that range, spend a jump and extend the boundary to the farthest seen.

JAVASCRIPT Copy

```
function jump(nums) {
  let jumps = 0, curEnd = 0, farthest = 0;
  for (let i = 0; i < nums.length - 1; i++) {
    farthest = Math.max(farthest, i + nums[i]);
    if (i === curEnd) {
      jumps++;
      curEnd = farthest;
    }
  }
  return jumps;
}
```

23.84 Task Scheduler

Approach: Greedily schedule the most frequent task first, laying out (maxFreq - 1) cooldown frames of size n+1 and appending the tasks that tie for the max, taking the larger of that arrangement and the raw task count.

JAVASCRIPT Copy

```
function leastInterval(tasks, n) {
  const counts = {};
  let maxFreq = 0;
  for (const t of tasks) {
    counts[t] = (counts[t] || 0) + 1;
    maxFreq = Math.max(maxFreq, counts[t]);
  }
```

```
  return Math.max(intervals, tasks.length);
}
```

23.85 Merge Intervals

JAVASCRIPT Copy

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const result = [intervals[0]];
  for (let i = 1; i < intervals.length; i++) {
    const last = result[result.length - 1];
    if (intervals[i][0] ≤ last[1]) last[1] = Math.max(last[1], intervals[
    else result.push(intervals[i]);
  }
  return result;
}
```

23.86 Non-overlapping Intervals

Approach: Sort by end time and greedily keep the interval that finishes earliest, counting any later interval that starts before the kept interval's end as a removal.

JAVASCRIPT Copy

```
function eraseOverlapIntervals(intervals) {
  if (intervals.length === 0) return 0;
  intervals.sort((a, b) => a[1] - b[1]);
  let count = 0, prevEnd = intervals[0][1];
  for (let i = 1; i < intervals.length; i++) {
    if (intervals[i][0] < prevEnd) {
      count++;
    } else {
      prevEnd = intervals[i][1];
    }
  }
```

23.87 Gas Station

Approach: If the total surplus of gas minus cost is non-negative a solution exists; greedily reset the start to the next station whenever the running tank drops below zero, since no station up to that point can be a valid start.

JAVASCRIPT Copy

```
function canCompleteCircuit(gas, cost) {
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
    const diff = gas[i] - cost[i];
    total += diff;
    tank += diff;
    if (tank < 0) {
      start = i + 1;
      tank = 0;
    }
  }
  return total ≥ 0 ? start : -1;
}
```

23.88 Assign Cookies

Approach: Sort both greed factors and cookie sizes ascending, then greedily give the smallest sufficient cookie to the least greedy remaining child.

JAVASCRIPT Copy

```
function findContentChildren(g, s) {
  g.sort((a, b) ⇒ a - b);
  s.sort((a, b) ⇒ a - b);
  let child = 0, cookie = 0;
  while (child < g.length && cookie < s.length) {
    if (s[cookie] ≥ g[child]) child++;
    cookie++;
  }
```

23.89 Partition Labels

JAVASCRIPT Copy

```
function partitionLabels(s) {
  const last = {};
  for (let i = 0; i < s.length; i++) last[s[i]] = i;
  const result = [];
  let start = 0, end = 0;
  for (let i = 0; i < s.length; i++) {
    end = Math.max(end, last[s[i]]);
    if (i === end) { result.push(end - start + 1); start = end + 1; }
  }
  return result;
}
```

23.90 Minimum Arrows to Burst Balloons

Approach: Sort balloons by end coordinate and greedily shoot an arrow at the current earliest end, only spending a new arrow when a balloon starts after that end.

JAVASCRIPT Copy

```
function findMinArrowShots(points) {
  if (points.length === 0) return 0;
  points.sort((a, b) => a[1] - b[1]);
  let arrows = 1, end = points[0][1];
  for (let i = 1; i < points.length; i++) {
    if (points[i][0] > end) {
      arrows++;
      end = points[i][1];
    }
  }
  return arrows;
}
```

QUICK CHECK — no marks, just for you.

Frontend System Desi...

1. According to the lesson, what is the pattern-recognition signal that a problem should be solved with a Greedy approach?

- The problem has overlapping subproblems whose results must be cached to avoid recomputation
- Every combination must be explored, backtracking whenever a branch fails
- The input must be scanned from both ends with two pointers moving toward each other
- At each step you can pick the locally optimal choice and it leads to the global optimum

Check answer

2. The lesson's Jump Game solution greedily tracks the farthest reachable index. What does this snippet print?

JAVASCRIPT Copy

```javascript
function canJump(nums) {
  let farthest = 0;
  for (let i = 0; i <= farthest && i < nums.length; i++) {
    farthest = Math.max(farthest, i + nums[i]);
  }
  return farthest >= nums.length - 1;
}

console.log(canJump([2, 3, 1, 1, 4]));
console.log(canJump([3, 2, 1, 0, 4]));
```

- true
  false
- false
  true

- false
  false

Check answer

3. This is the lesson's Partition Labels solution. What does it print?

JAVASCRIPT Copy

```javascript
function partitionLabels(s) {
  const last = {};
  for (let i = 0; i < s.length; i++) last[s[i]] = i;
  const result = [];
  let start = 0, end = 0;
  for (let i = 0; i < s.length; i++) {
    end = Math.max(end, last[s[i]]);
    if (i === end) { result.push(end - start + 1); start = end + 1; }
  }
  return result;
}

console.log(partitionLabels("ababcbacadefegdehijhklij").join(","));
```

- 9,8,7
- 8,7,9
- 9,7,8
- 10,7,7

Check answer

Dynamic programming

Next
Trees

LEARN MORE

ARTICLES & DOCS

Greedy algorithms (GeeksforGeeks)
geeksforgeeks.org

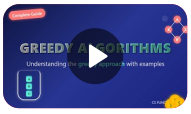
VIDEOS



Greedy algorithms explained
YouTube



Greedy in-depth for interviews
YouTube



Greedy for beginners
YouTube

Hand-picked references for this lesson.